Программирование на C

Лекция 15

Стандартная библиотека

Зачем нужна стандартная библиотека?

- Повторно используем готовые, документированные операции.
- Работаем с файлами, строками, памятью, числами и временем.
- Учимся читать **контракт функции**: параметры, результат, ограничения.
- Не всякая библиотечная функция безопасна при произвольных аргументах.

Сегодня: от объявления функции — к рабочему примеру.

Как читать сигнатуру?

```
char *fgets(char *restrict s, int n, FILE *restrict stream);
```

- `char *` перед именем — тип возвращаемого значения.
- `s`, `n`, `stream` — параметры.
- `const` запрещает изменение объекта через соответствующий указатель.
- `restrict` — специальное обещание об обращении к памяти; пока достаточно узнавать его в объявлениях.

Заголовочный файл предоставляет объявления; реализация обычно находится в библиотеке.

Заголовочные файлы

Заголовок	Основная область
`<stdio.h>`	потоки, файлы, форматированный ввод-вывод
`<stdlib.h>`	память, преобразования, сортировка, поиск
`<string.h>`	строки и блоки памяти
`<ctype.h>`	классификация символов
`<math.h>`	математика
`<time.h>`	время
`<assert.h>`, `<errno.h>`	проверки и диагностика

<stdio.h>

Ввод-вывод работает через объекты типа `FILE`.

Стандартные потоки: `stdin`, `stdout`, `stderr`.

Разберём: `printf`, `fprintf`, `scanf`, `fgets`, `fputs`, `fopen`, `fclose`, `fread`, `fwrite`.

`printf` — интерфейс

Заголовок: `<stdio.h>`

```
int printf(const char *restrict format, ...);
```

Назначение: Форматированный вывод в `stdout`.

Параметры: Строка формата и значения для спецификаторов `%d`, `%f`, `%s` и др.

Результат: Число выведенных символов или отрицательное значение при ошибке.

Важно: спецификатор обязан соответствовать типу аргумента.

`printf` — пример

```c
#include <stdio.h>
int main(void) {
    int count = 7;
    double mean = 2.5;
    printf("count=%d, mean=%.2f\n", count, mean);
}
```

`fprintf` — интерфейс

Заголовок: `<stdio.h>`

```
int fprintf(FILE *restrict stream,
            const char *restrict format, ...);
```

Назначение: Форматированный вывод в указанный поток.

Параметры: `stream` и формат с аргументами.

Результат: Число записанных символов или отрицательное значение.

В `stderr` удобно писать диагностические сообщения.

`fprintf` — пример

```c
#include <stdio.h>
int main(void) {
    fprintf(stdout, "Результат: %d\n", 42);
    fprintf(stderr, "Пример сообщения об ошибке\n");
}
```

`scanf` — интерфейс

Заголовок: `<stdio.h>`

```
int scanf(const char *restrict format, ...);
```

Назначение: Форматированный ввод из `stdin`.

Параметры: Формат и адреса объектов для записи.

Результат: Число успешно присвоенных элементов; `EOF` при ошибке ввода до первого присваивания.

Важно: всегда проверяем результат; `%s` без ограничения ширины опасен.

`scanf` — пример

```c
#include <stdio.h>
int main(void) {
    int age;
    if (scanf("%d", &age) == 1)
        printf("Возраст: %d\n", age);
    else
        fprintf(stderr, "Ожидалось целое число\n");
}
```

`fgets` — интерфейс

Заголовок: `<stdio.h>`

```
char *fgets(char *restrict s, int n,
            FILE *restrict stream);
```

Назначение: Читает не более `n - 1` символов и завершает строку `\0`.

Параметры: Буфер, его вместимость, поток.

Результат: `s` при успехе, `NULL` при ошибке или EOF без прочитанных символов.

Сохраняет `\n`, если он поместился; длинная строка может читаться частями.

`fgets` — пример

```
#include <stdio.h>
int main(void) {
    char line[64];
    if (fgets(line, sizeof line, stdin) != NULL)
        printf("Получено: %s", line);
}
```

`fputs` — интерфейс

Заголовок: `<stdio.h>`

```
int fputs(const char *restrict s,
          FILE *restrict stream);
```

Назначение: Записывает строку в поток без завершающего `\0`.

Параметры: Строка и поток.

Результат: Неотрицательное значение при успехе, `EOF` при ошибке.

Не добавляет перевод строки автоматически.

`fputs` — пример

```c
#include <stdio.h>
int main(void) {
    fputs("Первая строка\n", stdout);
    fputs("Вторая строка\n", stdout);
}
```

`fopen` — интерфейс

Заголовок: `<stdio.h>`

```
FILE *fopen(const char *restrict filename,
            const char *restrict mode);
```

Назначение: Открывает файл и создаёт поток.

Параметры: Путь и режим: `r`, `w`, `a`, `rb`, `wb` и др.

Результат: Указатель `FILE *` либо `NULL`.

`w` может обнулить существующий файл; `b` обозначает бинарный режим.

`fopen` — пример

```c
#include <stdio.h>
int main(void) {
    FILE *f = fopen("data.txt", "w");
    if (f == NULL) {
        perror("fopen");
        return 1;
    }
    fputs("Hello\n", f);
    fclose(f);
}
```

`fclose` — интерфейс

Заголовок: `<stdio.h>`

```
int fclose(FILE *stream);
```

Назначение: Закрывает поток, сбрасывая буферизированный вывод.

Параметры: Открытый поток.

Результат: `0` при успехе, `EOF` при ошибке.

После вызова указатель на закрытый поток использовать нельзя.

`fclose` — пример

```c
#include <stdio.h>
int main(void) {
    FILE *f = fopen("out.txt", "w");
    if (!f) return 1;
    fputs("42\n", f);
    if (fclose(f) == EOF) {
        perror("fclose");
        return 1;
    }
}
```

`fread` — интерфейс

Заголовок: `<stdio.h>`

```
size_t fread(void *restrict ptr, size_t size,
             size_t nmemb, FILE *restrict stream);
```

Назначение: Читает элементы фиксированного размера из потока.

Параметры: Буфер, размер элемента, число элементов, поток.

Результат: Количество полностью прочитанных элементов.

Короткое чтение может означать EOF или ошибку; проверяем `feof` / `ferror`.

`fread` — пример

```c
#include <stdio.h>
int main(void) {
    unsigned char data[16];
    FILE *f = fopen("data.bin", "rb");
    if (!f) return 1;
    size_t n = fread(data, 1, sizeof data, f);
    printf("Прочитано байтов: %zu\n", n);
    fclose(f);
}
```

`fwrite` — интерфейс

Заголовок: `<stdio.h>`

```
size_t fwrite(const void *restrict ptr, size_t size,
              size_t nmemb, FILE *restrict stream);
```

Назначение: Записывает элементы фиксированного размера.

Параметры: Буфер, размер элемента, число элементов, поток.

Результат: Количество полностью записанных элементов.

Бинарная запись объектов C не является переносимым форматом обмена между архитектурами.

`fwrite` — пример

```c
#include <stdio.h>
int main(void) {
    unsigned char data[] = {1, 2, 3, 4};
    FILE *f = fopen("data.bin", "wb");
    if (!f) return 1;
    size_t n = fwrite(data, 1, sizeof data, f);
    int ok = (n == sizeof data);
    if (fclose(f) == EOF) ok = 0;
    return ok ? 0 : 1;
}
```

`<stdio.h>` — что ещё полезно знать?

Функция	Назначение
`getchar`, `putchar`	один символ
`fscanf`	форматированный ввод из потока
`snprintf`	форматирование в буфер с ограничением длины
`feof`, `ferror`, `clearerr`	состояние потока
`perror`	диагностическое сообщение

Ошибка: проверять `feof` до чтения вместо проверки результата самой операции.

<string.h>

Строка C — последовательность символов с завершающим нулевым байтом `\0`.

Функции `mem*` работают с произвольными блоками памяти.

Разберём: `strlen`, `strcmp`, `strcpy`, `strchr`, `strstr`, `memcpy`, `memmove`, `memset`, `memcmp`.

`strlen` — интерфейс

Заголовок: `<string.h>`

```
size_t strlen(const char *s);
```

Назначение: Определяет длину строки до первого `\0`.

Параметры: Указатель на корректную нуль-терминированную строку.

Результат: Количество символов до `\0` (без него).

Не знает размер выделенного буфера; отсутствие `\0` ведёт к неопределённому поведению.

`strlen` — пример

```c
#include <stdio.h>
#include <string.h>
int main(void) {
    char word[] = "hello";
    printf("Длина: %zu\n", strlen(word));
    printf("Размер массива: %zu\n", sizeof word);
}
```

`strcmp` — интерфейс

Заголовок: `<string.h>`

```
int strcmp(const char *s1, const char *s2);
```

Назначение: Лексикографически сравнивает две строки.

Параметры: Две корректные строки.

Результат: Отрицательное, нулевое или положительное число.

Не следует рассчитывать, что результат равен именно `-1` или `1`.

`strcmp` — пример

```c
#include <stdio.h>
#include <string.h>
int main(void) {
    const char *a = "apple";
    const char *b = "banana";
    if (strcmp(a, b) < 0)
        puts("apple идёт раньше banana");
}
```

`strcpy` — интерфейс

Заголовок: `<string.h>`

```
char *strcpy(char *restrict dest,
             const char *restrict src);
```

Назначение: Копирует строку вместе с `\0`.

Параметры: Буфер назначения достаточной вместимости и строка-источник.

Результат: `dest`.

Опасность: функция не знает размер `dest`; перекрытие областей недопустимо.

`strcpy` — пример

```c
#include <stdio.h>
#include <string.h>
int main(void) {
    char dest[16];
    const char *src = "hello";
    if (strlen(src) + 1 <= sizeof dest) {
        strcpy(dest, src);
        puts(dest);
    }
}
```

`strchr` — интерфейс

Заголовок: `<string.h>`

```
char *strchr(const char *s, int c);
```

Назначение: Ищет первое вхождение символа в строке.

Параметры: Строка и код искомого символа.

Результат: Указатель на найденный символ или `NULL`.

Может найти и завершающий `\0`.

`strchr` — пример

```c
#include <stdio.h>
#include <string.h>
int main(void) {
    char text[] = "key=value";
    char *equal = strchr(text, '=');
    if (equal != NULL)
        printf("Значение: %s\n", equal + 1);
}
```

`strstr` — интерфейс

Заголовок: `<string.h>`

```
char *strstr(const char *haystack,
             const char *needle);
```

Назначение: Ищет первое вхождение подстроки.

Параметры: Строка и искомая подстрока.

Результат: Указатель на начало вхождения или `NULL`.

Пустая подстрока находится в начале строки.

`strstr` — пример

```c
#include <stdio.h>
#include <string.h>
int main(void) {
    const char *text = "programming in C";
    const char *pos = strstr(text, "in C");
    if (pos != NULL)
        printf("Позиция: %td\n", pos - text);
}
```

`memcpy` — интерфейс

Заголовок: `<string.h>`

```
void *memcpy(void *restrict dest,
             const void *restrict src, size_t n);
```

Назначение: Копирует `n` байтов.

Параметры: Адреса двух неперекрывающихся областей и число байтов.

Результат: `dest`.

Если области перекрываются — неопределённое поведение; используем `memmove`.

`memcpy` — пример

```c
#include <stdio.h>
#include <string.h>
int main(void) {
    int a[] = {3, 1, 4};
    int b[3];
    memcpy(b, a, sizeof a);
    printf("%d %d %d\n", b[0], b[1], b[2]);
}
```

`memmove` — интерфейс

Заголовок: `<string.h>`

```
void *memmove(void *dest,
              const void *src, size_t n);
```

Назначение: Копирует `n` байтов, в том числе при перекрытии областей.

Параметры: Адреса областей и число байтов.

Результат: `dest`.

Удобна для сдвига элементов внутри массива.

`memmove` — пример

```
#include <stdio.h>
#include <string.h>
int main(void) {
    int a[5] = {1, 2, 3, 4, 0};
    memmove(a + 2, a + 1, 3 * sizeof a[0]);
    a[1] = 99;
    for (size_t i = 0; i < 5; ++i)
        printf("%d ", a[i]);
}
```

`memset` — интерфейс

Заголовок: `<string.h>`

```
void *memset(void *s, int c, size_t n);
```

Назначение: Заполняет `n` байтов одним байтовым значением.

Параметры: Буфер, значение байта, число байтов.

Результат: `s`.

Безопасно для обнуления массива `unsigned char`; не универсальный способ присвоить любое значение элементам произвольного типа.

`memset` — пример

```c
#include <stdio.h>
#include <string.h>
int main(void) {
    unsigned char bytes[8];
    memset(bytes, 0, sizeof bytes);
    bytes[3] = 7;
    printf("%u\n", (unsigned)bytes[3]);
}
```

`memcmp` — интерфейс

Заголовок: `<string.h>`

```
int memcmp(const void *s1,
           const void *s2, size_t n);
```

Назначение: Сравнивает первые `n` байтов двух блоков.

Параметры: Два доступных блока и число байтов.

Результат: Отрицательное, нулевое или положительное число.

Сравнивает **представления**, а не логические значения структур (возможны padding-байты).

`memcmp` — пример

```c
#include <stdio.h>
#include <string.h>
int main(void) {
    unsigned char a[] = {1, 2, 3};
    unsigned char b[] = {1, 2, 4};
    if (memcmp(a, b, sizeof a) < 0)
        puts("a лексикографически меньше b");
}
```

`<string.h>` — дополнительные функции

- `strncmp` — сравнение не более `n` символов.
- `strcat`, `strncat` — конкатенация (следим за вместимостью буфера).
- `strrchr` — последнее вхождение символа.
- `strtok` — разбиение строки на токены; **изменяет исходный буфер** и хранит внутреннее состояние.

Для лабораторной по CSV особенно важно понимать, почему `strtok` не заменяет полноценный CSV-парсер.

<stdlib.h>

Преобразование строк, динамическая память, сортировка и поиск.

Разберём: `malloc`, `calloc`, `realloc`, `free`, `strtol`, `strtod`, `qsort`, `bsearch`, `rand`, `srand`.

`malloc` — интерфейс

Заголовок: `<stdlib.h>`

```
void *malloc(size_t size);
```

Назначение: Выделяет блок динамической памяти.

Параметры: Требуемый размер в байтах.

Результат: Указатель на блок или `NULL`.

Содержимое не инициализировано; память освобождаем через `free`.

`malloc` — пример

```c
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    size_t n = 5;
    int *a = malloc(n * sizeof *a);
    if (a == NULL) return 1;
    for (size_t i = 0; i < n; ++i) a[i] = (int)i;
    printf("%d\n", a[4]);
    free(a);
}
```

`calloc` — интерфейс

Заголовок: `<stdlib.h>`

```
void *calloc(size_t nmemb, size_t size);
```

Назначение: Выделяет память для массива и обнуляет все байты.

Параметры: Число элементов и размер элемента.

Результат: Указатель или `NULL`.

Байтовое обнуление не следует отождествлять с присваиванием нулевых значений всем возможным типам.

`calloc` — пример

```c
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    size_t n = 10;
    int *counts = calloc(n, sizeof *counts);
    if (counts == NULL) return 1;
    printf("%d\n", counts[0]);
    free(counts);
}
```

`realloc` — интерфейс

Заголовок: `<stdlib.h>`

```
void *realloc(void *ptr, size_t size);
```

Назначение: Изменяет размер ранее выделенного блока, при необходимости перемещая его.

Параметры: Старый указатель (`NULL` допустим) и новый размер.

Результат: Новый указатель либо `NULL` при неудаче.

При ошибке старый блок остаётся действительным; используем временный указатель. Избегаем неоднозначного случая `size == 0`.

`realloc` — пример

```c
#include <stdlib.h>
int main(void) {
    size_t capacity = 4;
    int *a = malloc(capacity * sizeof *a);
    if (!a) return 1;
    size_t new_capacity = 8;
    int *tmp = realloc(a, new_capacity * sizeof *a);
    if (!tmp) { free(a); return 1; }
    a = tmp;
    capacity = new_capacity;
    free(a);
}
```

`free` — интерфейс

Заголовок: `<stdlib.h>`

```
void free(void *ptr);
```

Назначение: Освобождает ранее выделенный блок.

Параметры: Указатель, возвращённый функцией выделения памяти, либо `NULL`.

Результат: Ничего (`void`).

Повторный `free`, освобождение невыделенного адреса и обращение после `free` — ошибки.

`free` — пример

```c
#include <stdlib.h>
int main(void) {
    int *p = malloc(sizeof *p);
    if (!p) return 1;
    *p = 42;
    free(p);
    p = NULL;
    free(p);  // допустимо
}
```

`strtol` — интерфейс

Заголовок: `<stdlib.h>`

```
long strtol(const char *restrict nptr,
            char **restrict endptr, int base);
```

Назначение: Преобразует начало строки в целое число.

Параметры: Строка, указатель для позиции окончания, основание `2..36` или `0`.

Результат: Число типа `long`; ошибки диапазона через `errno`.

Для строгой проверки убеждаемся, что цифры были, остаток строки пуст и не было `ERANGE`.

`strtol` — пример

```c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
int main(void) {
    char text[] = "123x";
    char *end;
    errno = 0;
    long x = strtol(text, &end, 10);
    if (end == text || *end != '\0' || errno == ERANGE)
        puts("Некорректное целое число");
    else
        printf("%ld\n", x);
}
```

`strtod` — интерфейс

Заголовок: `<stdlib.h>`

```
double strtod(const char *restrict nptr,
              char **restrict endptr);
```

Назначение: Преобразует начало строки в число `double`.

Параметры: Строка и указатель для окончания разбора.

Результат: Число `double`; ошибки диапазона через `errno`.

Допускает синтаксис вроде `1.2e3`, а также `inf` / `nan`; формат зависит от locale.

`strtod` — пример

```c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
int main(void) {
    char text[] = "3.125";
    char *end;
    errno = 0;
    double x = strtod(text, &end);
    if (end != text && *end == '\0' && errno != ERANGE)
        printf("%.3f\n", x);
}
```

`qsort` — интерфейс

Заголовок: `<stdlib.h>`

```
void qsort(void *base, size_t nmemb, size_t size,
           int (*compar)(const void *, const void *));
```

Назначение: Сортирует массив по компаратору.

Параметры: Массив, число элементов, размер элемента, функция сравнения.

Результат: Ничего (`void`).

Компаратор возвращает знак сравнения; нельзя полагаться на стабильность сортировки.

`qsort` — пример

```c
#include <stdio.h>
#include <stdlib.h>
int cmp_int(const void *pa, const void *pb) {
    int a = *(const int *)pa, b = *(const int *)pb;
    return (a > b) - (a < b);
}
int main(void) {
    int a[] = {7, 2, 9, 1};
    size_t n = sizeof a / sizeof a[0];
    qsort(a, n, sizeof a[0], cmp_int);
    for (size_t i = 0; i < n; ++i)
        printf("%d ", a[i]);
}
```

`bsearch` — интерфейс

Заголовок: `<stdlib.h>`

```
void *bsearch(const void *key, const void *base,
              size_t nmemb, size_t size,
              int (*compar)(const void *, const void *));
```

Назначение: Ищет элемент в отсортированном массиве.

Параметры: Ключ, массив, число и размер элементов, компаратор.

Результат: Указатель на найденный элемент или `NULL`.

Массив должен быть упорядочен по тому же отношению, которое задаёт компаратор.

`bsearch` — пример

```c
#include <stdio.h>
#include <stdlib.h>
int cmp_int(const void *pa, const void *pb) {
    int a = *(const int *)pa, b = *(const int *)pb;
    return (a > b) - (a < b);
}
int main(void) {
    int a[] = {1, 2, 7, 9};
    int key = 7;
    int *p = bsearch(&key, a, 4, sizeof a[0], cmp_int);
    if (p != NULL) printf("Найдено: %d\n", *p);
}
```

`rand / srand` — интерфейс

Заголовок: `<stdlib.h>`

```
int rand(void);
void srand(unsigned int seed);
```

Назначение: `rand` выдаёт псевдослучайное число; `srand` задаёт начальное состояние генератора.

Параметры: Для `srand` — начальное значение (`seed`).

Результат: `rand`: целое от `0` до `RAND_MAX`; `srand`: ничего.

Не использовать для криптографии; `rand() % n` может давать смещение распределения.

`rand / srand` — пример

```c
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    srand(12345);  // воспроизводимая последовательность
    for (int i = 0; i < 5; ++i)
        printf("%d\n", rand());
}
```

`<stdlib.h>` — завершение программы

Функция	Действие
`exit(status)`	нормальное завершение с обработчиками и закрытием потоков
`abort()`	аварийное завершение
`atexit(handler)`	регистрация обработчика завершения

Константы `EXIT_SUCCESS` и `EXIT_FAILURE` задают переносимые статусы.

Не путать: `return` из `main` и `return` из обычной функции.

<ctype.h>

Классификация и преобразование символов.

Разберём: `isdigit`, `isalpha`, `isspace`, `tolower`, `toupper`.

`isdigit / isalpha` — интерфейс

Заголовок: `<ctype.h>`

```
int isdigit(int c);
int isalpha(int c);
```

Назначение: Проверяют, является ли символ десятичной цифрой или буквой.

Параметры: `EOF` либо значение, представимое как `unsigned char`.

Результат: Ноль при ложном результате, ненулевое значение при истинном.

Для потенциально отрицательного `char` сначала приводим к `unsigned char`.

`isdigit / isalpha` — пример

```c
#include <stdio.h>
#include <ctype.h>
int main(void) {
    char text[] = "A7";
    for (size_t i = 0; text[i] != '\0'; ++i) {
        unsigned char c = (unsigned char)text[i];
        printf("%d %d\n", !!isalpha(c), !!isdigit(c));
    }
}
```

`isspace` — интерфейс

Заголовок: `<ctype.h>`

```
int isspace(int c);
```

Назначение: Проверяет пробельный символ (пробел, табуляция, перевод строки и др.).

Параметры: `EOF` либо значение `unsigned char`.

Результат: Ноль или ненулевое значение.

Набор пробельных символов не ограничивается обычным пробелом `' '`.

`isspace` — пример

```c
#include <stdio.h>
#include <ctype.h>
int main(void) {
    char text[] = "  42\t";
    size_t i = 0;
    while (isspace((unsigned char)text[i])) ++i;
    printf("Первый непробельный: %c\n", text[i]);
}
```

`tolower / toupper` — интерфейс

Заголовок: `<ctype.h>`

```
int tolower(int c);
int toupper(int c);
```

Назначение: Преобразуют букву в нижний или верхний регистр.

Параметры: `EOF` либо значение `unsigned char`.

Результат: Преобразованный символ или исходное значение.

Поведение для букв зависит от текущей локали; не является универсальной Unicode-конвертацией.

`tolower / toupper` — пример

```c
#include <stdio.h>
#include <ctype.h>
int main(void) {
    char text[] = "HeLLo";
    for (size_t i = 0; text[i] != '\0'; ++i)
        text[i] = (char)tolower((unsigned char)text[i]);
    puts(text);
}
```

<math.h>

Математические функции обычно работают с числами с плавающей точкой.

Разберём: `sqrt`, `pow`, `sin`, `cos`, `exp`, `log`, `fabs`, `floor`, `ceil`, `round`.

`sqrt / pow` — интерфейс

Заголовок: `<math.h>`

```
double sqrt(double x);
double pow(double x, double y);
```

Назначение: Квадратный корень и возведение в степень.

Параметры: Аргументы типа `double`.

Результат: Результат типа `double`.

Следим за областью определения и ошибками диапазона; `pow(x, 2)` не всегда лучший способ вычислить `x*x`.

`sqrt / pow` — пример

```c
#include <stdio.h>
#include <math.h>
int main(void) {
    double x = 9.0;
    printf("sqrt=%.1f\n", sqrt(x));
    printf("pow=%.1f\n", pow(2.0, 5.0));
}
```

Для GCC/Clang на некоторых системах требуется линковка с `-lm`.

`sin / cos` — интерфейс

Заголовок: `<math.h>`

```
double sin(double x);
double cos(double x);
```

Назначение: Тригонометрические функции.

Параметры: Угол в **радианах**.

Результат: Значение типа `double`.

Не передавайте градусы без преобразования.

`sin / cos` — пример

```c
#include <stdio.h>
#include <math.h>
int main(void) {
    double pi = acos(-1.0);
    double angle = pi / 3.0;
    printf("sin=%.3f\n", sin(angle));
    printf("cos=%.3f\n", cos(angle));
}
```

`exp / log` — интерфейс

Заголовок: `<math.h>`

```
double exp(double x);
double log(double x);
```

Назначение: Экспонента и натуральный логарифм.

Параметры: Аргумент типа `double`; для `log` требуется `x > 0` для конечного вещественного результата.

Результат: Значение типа `double`.

`log` — натуральный логарифм; для десятичного есть `log10`.

`exp / log` — пример

```c
#include <stdio.h>
#include <math.h>
int main(void) {
    double x = 2.0;
    printf("exp(log(x)) = %.6f\n", exp(log(x)));
}
```

`fabs` — интерфейс

Заголовок: `<math.h>`

```
double fabs(double x);
```

Назначение: Возвращает абсолютную величину вещественного числа.

Параметры: Число типа `double`.

Результат: Неотрицательное значение типа `double` (или NaN).

Для целых типов есть `abs` и `labs` из `<stdlib.h>`.

`fabs` — пример

```c
#include <stdio.h>
#include <math.h>
int main(void) {
    double error = -0.125;
    printf("Модуль ошибки: %.3f\n", fabs(error));
}
```

`floor / ceil / round` — интерфейс

Заголовок: `<math.h>`

```
double floor(double x);
double ceil(double x);
double round(double x);
```

Назначение: Округление вниз, вверх и до ближайшего целого (половины — от нуля).

Параметры: Число типа `double`.

Результат: Результат типа `double`, не `int`.

`round` отличается от приведения `(int)x`, которое отбрасывает дробную часть.

`floor / ceil / round` — пример

```c
#include <stdio.h>
#include <math.h>
int main(void) {
    double x = -2.5;
    printf("floor: %.1f\n", floor(x)); // -3
    printf("ceil:  %.1f\n", ceil(x));  // -2
    printf("round: %.1f\n", round(x)); // -3
}
```

Компоновка с математической библиотекой

На некоторых Unix-подобных системах при использовании `<math.h>` требуется `-lm`:

```
gcc -std=c23 -Wall -Wextra main.c -lm -o program
```

- `#include <math.h>` — объявления функций.
- `-lm` — подключение библиотеки при компоновке.
- Требование `-lm` зависит от компилятора и платформы.

<time.h>

Календарное время и время процессора — разные величины.

Разберём: `time`, `difftime`, `clock`.

`time` — интерфейс

Заголовок: `<time.h>`

```c
time_t time(time_t *timer);
```

Назначение: Получает текущее календарное время.

Параметры: Указатель для записи результата или `NULL`.

Результат: Значение `time_t` либо `(time_t)-1` при ошибке.

Единицы и представление `time_t` не следует считать универсально равными секундам от Unix epoch.

`time` — пример

```c
#include <stdio.h>
#include <time.h>
int main(void) {
    time_t now = time(NULL);
    if (now == (time_t)-1) return 1;
    printf("Время получено\n");
}
```

`difftime` — интерфейс

Заголовок: `<time.h>`

```
double difftime(time_t time1, time_t time0);
```

Назначение: Вычисляет разницу между двумя календарными моментами.

Параметры: Два значения `time_t`.

Результат: Разница в секундах типа `double`.

Используем вместо арифметического вычитания `time_t`.

`difftime` — пример

```c
#include <stdio.h>
#include <time.h>
int main(void) {
    time_t start = time(NULL);
    if (start == (time_t)-1) return 1;
    /* ... некоторая работа ... */
    time_t finish = time(NULL);
    if (finish == (time_t)-1) return 1;
    printf("%.0f секунд\n", difftime(finish, start));
}
```

`clock` — интерфейс

Заголовок: `<time.h>`

```
clock_t clock(void);
```

Назначение: Получает оценку процессорного времени, использованного программой.

Параметры: Нет параметров.

Результат: `clock_t` или `(clock_t)-1`, если значение недоступно.

Для секунд делим разность на `CLOCKS_PER_SEC`; это **не** обязательно реальное прошедшее время.

`clock` — пример

```
#include <stdio.h>
#include <time.h>
int main(void) {
    clock_t start = clock();
    if (start == (clock_t)-1) return 1;
    volatile unsigned long sum = 0;
    for (unsigned long i = 0; i < 1000000UL; ++i)
        sum = sum + i;
    clock_t finish = clock();
    if (finish == (clock_t)-1) return 1;
    printf("%.6f c CPU\n",
        (double)(finish - start) / CLOCKS_PER_SEC);
}
```

<assert.h> и <errno.h>

Проверка внутренних предположений и диагностика ошибок.

Разберём: `assert`, `perror`.

`assert` — интерфейс

Заголовок: `<assert.h>`

```
#include <assert.h>
assert(expression);
```

Назначение: Проверяет программное предположение; при ложном выражении обычно аварийно завершает программу.

Параметры: Скалярное выражение.

Результат: Не функция: макрос, значения не возвращает.

При определённом `NDEBUG` проверки могут быть отключены. Не помещайте внутрь необходимые побочные эффекты.

`assert` — пример

```c
#include <assert.h>
int divide(int a, int b) {
    assert(b != 0);
    return a / b;
}
int main(void) {
    return divide(12, 3) != 4;
}
```

`perror / errno` — интерфейс

Заголовок: `<stdio.h>, <errno.h>`

```
void perror(const char *s);
/* errno — изменяемый макрос-lvalue типа int */
```

Назначение: `errno` сообщает код некоторых ошибок библиотеки; `perror` выводит описание текущего кода.

Параметры: Для `perror` — префикс сообщения.

Результат: `perror` ничего не возвращает.

Не всякая функция устанавливает `errno`; читать его нужно только по документированному контракту.

`perror / errno` — пример

```c
#include <stdio.h>
#include <errno.h>
int main(void) {
    FILE *f = fopen("missing-file.txt", "r");
    if (f == NULL) {
        int saved_errno = errno;
        errno = saved_errno;
        perror("fopen");
        return 1;
    }
    fclose(f);
}
```

`size_t`, `NULL`, `EOF` — это не функции

- `size_t` — беззнаковый целочисленный тип для размеров объектов; печать через `%zu`.
- `NULL` — макрос для нулевого указателя.
- `EOF` — отрицательная целочисленная константа для обозначения конца ввода или ошибки в ряде функций.
- `EXIT_SUCCESS`, `EXIT_FAILURE` — статусы завершения.
- `RAND_MAX` — верхняя граница результата `rand()`.

Их объявления и определения находятся в соответствующих стандартных заголовках.

`void *` — универсальный указатель на объект

```
void *malloc(size_t size);
void *bsearch(const void *key, const void *base,
              size_t n, size_t size,
              int (*cmp)(const void *, const void *));
```

- `void *` можно преобразовать к указателю на объектный тип.
- Разыменовать `void *` напрямую нельзя.
- `const void *` запрещает изменение объекта через этот указатель.
- Указатель на функцию — отдельная категория: он не является `void *`.

Проверяем контракты: мини-задачи

1. Почему `realloc(p, n)` нельзя сразу присваивать `p`?
2. Почему `strcmp(a, b) == -1` — плохая проверка?
3. Чем `memcpy` отличается от `memmove`?
4. Почему нельзя использовать `sizeof(ptr)` как размер выделенного массива?
5. Что произойдёт, если `qsort` получает некорректный компаратор?
6. Почему `clock()` не измеряет строго время по настенным часам?

Где искать точные сведения?

- **Стандарт ISO C** — нормативное описание.
- Справочник cppreference (раздел C) — быстрый обзор.
- `man 3 printf`, `man 3 malloc` — документация конкретной системы.
- Заголовочные файлы — объявления, но не полная спецификация поведения.

Главный навык: прочитать контракт, проверить результат и учесть ограничения.